

CO3 AT3 (DATA INTEGRATION TEST) CHOOSE 3 QUESTIONS

Name : Vedvyass M

Reg no : 192324267

1) Question 1 – Key Identification and Structural Inconsistencies

Aim

To inspect multiple student placement datasets, identify the most reliable common key, detect structural inconsistencies, standardize the key fields, and determine the most appropriate identifier for data integration.

Problem Analysis

The college placement department maintains separate datasets containing Student Details, Academic Performance, Technical Skills, Attendance, and Placement Information.

Different datasets may use different column names such as:

- Student_ID
- ID
- StudentID

Some records may also contain missing or duplicated identifiers. Therefore, the datasets must be inspected and standardized before integration.

Dataset Inspection

First, all datasets are loaded using Pandas and their structures are examined.

```
import pandas as pd
```

```
students = pd.read_csv("Student_Details.csv")  
academic = pd.read_csv("Academic_Performance.csv")  
skills = pd.read_csv("Technical_Skills.csv")  
attendance = pd.read_csv("Attendance.csv")  
placement = pd.read_csv("Placement.csv")
```

```

datasets = {
    "Student Details": students,
    "Academic": academic,
    "Skills": skills,
    "Attendance": attendance,
    "Placement": placement
}

for name, df in datasets.items():
    print("\n---", name, "---")
    print("Rows:", df.shape[0])
    print("Columns:", df.shape[1])
    print("Column Names:", list(df.columns))
    print(df.dtypes)
    print("Missing Values:")
    print(df.isnull().sum())

```

Identifying the Common Key

The common identifier is expected to be represented differently in different files.

For example:

```

Student Details    → Student_ID
Academic Performance → ID
Technical Skills    → StudentID
Attendance          → Student_ID
Placement          → ID

```

These columns represent the same entity and must therefore be standardized.

```

students.rename(columns={"ID": "Student_ID"}, inplace=True)
academic.rename(columns={"ID": "Student_ID"}, inplace=True)
skills.rename(columns={"StudentID": "Student_ID"}, inplace=True)
attendance.rename(columns={"ID": "Student_ID"}, inplace=True)
placement.rename(columns={"ID": "Student_ID"}, inplace=True)

```

Standardizing the Key

The identifier should have the same data type and formatting in every dataset.

```

for df in [students, academic, skills, attendance, placement]:
    df["Student_ID"] = (
        df["Student_ID"]
        .astype(str)
        .str.strip()
    )

```

Checking Missing and Duplicate Keys

```

for name, df in datasets.items():
    print("\n", name)

    print("Missing Student_ID:",
          df["Student_ID"].isnull().sum())

```

```
print("Duplicate Student_ID:",  
      df["Student_ID"].duplicated().sum())
```

Missing identifiers should not be automatically deleted. They should first be investigated because removing them may result in loss of valid student information.

Why Student_ID Is the Most Appropriate Key

Student_ID is the most reliable integration key because it is designed to uniquely identify a student.

Using the student's **Name** is inappropriate because:

- Two students can have the same name.
- Names may have different spellings.
- Names may contain extra spaces.
- Names can change.
- Names are not guaranteed to be unique.

Using **Department** is also inappropriate because:

- Many students belong to the same department.
- Department does not uniquely identify an individual.
- Students may change departments or programs.

Therefore, the integration should be based on Student_ID.

Validation

```
print("Student Details unique IDs:",  
      students["Student_ID"].nunique())
```

```
print("Academic unique IDs:",  
      academic["Student_ID"].nunique())
```

```
print("Skills unique IDs:",  
      skills["Student_ID"].nunique())
```

```
print("Attendance unique IDs:",  
      attendance["Student_ID"].nunique())
```

```
print("Placement unique IDs:",  
      placement["Student_ID"].nunique())
```

Conclusion

The inspection shows that the datasets contain structural inconsistencies such as different column names and potentially different data types. These inconsistencies must be corrected before integration. Student_ID is selected as the common integration key because it is a unique student-level identifier, whereas Name and Department cannot reliably distinguish individual students.

2) Appropriate Join Strategy

Aim

To integrate Student Details, Academic Performance, Technical Skills, and Placement Information while ensuring that all 1,000 registered students remain in the final dataset.

Problem Analysis

The Student Details dataset contains **1,000 registered students**, whereas the Academic Performance dataset contains only **960 records**.

Therefore:

```
Registered students = 1,000
Academic records    = 960
Missing records     = 1,000 - 960
                   = 40
```

The objective is to retain every registered student, including students who do not have academic information.

Loading the Datasets

```
import pandas as pd
```

```
students = pd.read_csv("Student_Details.csv")
academic = pd.read_csv("Academic_Performance.csv")
skills = pd.read_csv("Technical_Skills.csv")
placement = pd.read_csv("Placement.csv")
```

Standardizing Student_ID

```
students.rename(columns={"ID": "Student_ID"}, inplace=True)
academic.rename(columns={"ID": "Student_ID"}, inplace=True)
skills.rename(columns={"StudentID": "Student_ID"}, inplace=True)
placement.rename(columns={"ID": "Student_ID"}, inplace=True)
```

```

for df in [students, academic, skills, placement]:
    df["Student_ID"] = (
        df["Student_ID"]
        .astype(str)
        .str.strip()
    )

```

Identifying the 40 Missing Academic Records

```

missing_academic = students[
    ~students["Student_ID"].isin(
        academic["Student_ID"]
    )
]

print(missing_academic)
print("Number of missing academic records:",
      len(missing_academic))

```

The expected result is:

Number of missing academic records: 40

These 40 students are registered students for whom no matching academic record exists.

Selecting the Join Strategy

The **Student Details dataset is the master dataset** because it contains all registered students.

Therefore, a **LEFT JOIN** is appropriate.

Integration

```

integrated = students.merge(
    academic,
    on="Student_ID",
    how="left"
)

print("After Academic merge:", integrated.shape)

integrated = integrated.merge(
    skills,
    on="Student_ID",
    how="left"
)

print("After Skills merge:", integrated.shape)

integrated = integrated.merge(
    placement,
    on="Student_ID",
    how="left"
)

```

)

```
print("After Placement merge:", integrated.shape)
```

Why LEFT JOIN?

A LEFT JOIN keeps every record from the Student Details dataset.

For example:

Student Details = 1,000
Academic = 960

LEFT JOIN

↓
All 1,000 students retained

↓
40 students have missing academic values

An INNER JOIN would retain only students having matching academic records and could remove the 40 students.

Validation

```
print("Original unique students:",  
      students["Student_ID"].nunique())
```

```
print("Final unique students:",  
      integrated["Student_ID"].nunique())
```

```
print("Final rows:",  
      len(integrated))
```

```
print("\nMissing values:")  
print(integrated.isnull().sum())
```

Handling Missing Values

The 40 students should **not be deleted** simply because academic information is unavailable.

Their academic fields can remain as NaN and the missingness should be documented.

For example:

Student_ID	Name	CGPA	Attendance
S1001	Student A	8.2	92
S1002	Student B	NaN	85

The NaN indicates that academic information was unavailable rather than indicating that the student does not exist.

Conclusion

A LEFT JOIN is the appropriate strategy because the requirement is to retain all 1,000 registered students. The Academic dataset contains only 960 records, resulting in 40 students without academic information. These students should remain in the integrated dataset with missing academic values. The decision is based on the analytical requirement rather than simply selecting the join that produces more records.

3) Systematic Integration Validation

Aim

To design a systematic validation procedure for an integrated student placement dataset and identify problems related to row counts, duplicate identifiers, missing values, data types, and unmatched records.

Problem Analysis

A dataset can be successfully merged using Pandas without producing any programming error while still containing incorrect results.

For example, the final dataset may contain:

- Duplicate students
- Unexpected missing values
- Lost placement records
- Duplicate keys
- Incorrect data types
- Unmatched records

Therefore, validation must be performed after every major merge.

Step 1 – Load the Datasets

```
import pandas as pd
```

```
students = pd.read_csv("Student_Details.csv")  
academic = pd.read_csv("Academic_Performance.csv")  
skills = pd.read_csv("Technical_Skills.csv")  
placement = pd.read_csv("Placement.csv")
```

Step 2 – Check Row Counts

```
print("Student rows:", len(students))
```

```
print("Academic rows:", len(academic))
print("Skills rows:", len(skills))
print("Placement rows:", len(placement))
```

After each merge:

```
integrated = students.merge(
    academic,
    on="Student_ID",
    how="left"
)

print("After Academic merge:", len(integrated))

integrated = integrated.merge(
    skills,
    on="Student_ID",
    how="left"
)

print("After Skills merge:", len(integrated))

integrated = integrated.merge(
    placement,
    on="Student_ID",
    how="left"
)

print("After Placement merge:", len(integrated))
```

Why this matters

A sudden and unexpected increase in rows may indicate duplicate keys or a one-to-many relationship.

A sudden decrease may indicate an inappropriate join, such as using an INNER JOIN when all students were required.

Step 3 – Check Unique Student IDs

```
print("Total rows:",
      len(integrated))

print("Unique Student IDs:",
      integrated["Student_ID"].nunique())
```

If:

Rows = 1,275
Unique Student IDs = 1,000

then some students appear more than once.

This does not automatically mean the records are erroneous because one student may have multiple placement opportunities.

Step 4 – Check Duplicate Keys

```
duplicate_ids = integrated[
    integrated["Student_ID"].duplicated(
        keep=False
    )
]

print(duplicate_ids)
```

The number of duplicate students can be calculated using:

```
print(
    integrated["Student_ID"]
    .value_counts()
    .loc[lambda x: x > 1]
)
```

This helps determine whether duplicate records are caused by data errors or legitimate one-to-many relationships.

Step 5 – Check Missing Values

```
missing = integrated.isnull().sum()

print(missing)
```

The percentage of missing values can also be calculated:

```
missing_percentage = (
    integrated.isnull().mean() * 100
)

print(missing_percentage)
```

[Why this matters](#)

Unexpected missing values after a merge may indicate that records from one dataset did not match the integration key.

However, some missing values may be legitimate, such as students who have not yet received a placement.

Step 6 – Check Data Types

```
print(integrated.dtypes)
```

Important columns should have appropriate types.

For example:

```
integrated["CGPA"] = pd.to_numeric(
    integrated["CGPA"],
```

```

        errors="coerce"
    )

    integrated["Attendance"] = pd.to_numeric(
        integrated["Attendance"],
        errors="coerce"
    )

```

Why this matters

If CGPA is stored as text instead of numeric data, calculations such as average CGPA may produce incorrect results.

Step 7 – Check Unmatched Student Records

To find academic records that do not exist in Student Details:

```

academic_unmatched = academic[
    ~academic["Student_ID"].isin(
        students["Student_ID"]
    )
]

print("Unmatched academic records:",
      len(academic_unmatched))

```

To find placement records without a corresponding student:

```

placement_unmatched = placement[
    ~placement["Student_ID"].isin(
        students["Student_ID"]
    )
]

print("Unmatched placement records:",
      len(placement_unmatched))

```

These records should be investigated rather than automatically deleted.

Step 8 – Validate Missing Placement Information

```

students_without_placement = students[
    ~students["Student_ID"].isin(
        placement["Student_ID"]
    )
]

print(
    "Students without placement:",
    len(students_without_placement)
)

```

This may be completely legitimate because not every registered student necessarily receives a placement offer.

Step 9 – Final Validation Summary

```
print("===== FINAL VALIDATION =====")

print("Final rows:",
      len(integrated))

print("Unique students:",
      integrated["Student_ID"].nunique())

print("Duplicate IDs:",
      integrated["Student_ID"].duplicated().sum())

print("Total missing values:",
      integrated.isnull().sum().sum())

print("Data types:")
print(integrated.dtypes)
```

Validation Interpretation

Validation	Problem Detected
Row count	Lost or unexpectedly added records
Unique IDs	Incorrect student count
Duplicate keys	Duplicate or one-to-many relationships
Missing values	Unmatched or incomplete information
Data types	Incorrect numerical/date/text representation
Unmatched records	Records existing in only one dataset

Export Final Dataset

After successful validation:

```
integrated.to_csv(
    "Final_Integrated_Dataset.csv",
    index=False
)

print("Final_Integrated_Dataset.csv created successfully.")
```

Conclusion

Successful execution of a Pandas merge does not guarantee that the integrated dataset is correct. Systematic validation is required to verify row counts, unique identifiers, duplicate keys, missing values, data types, and unmatched records. Duplicate students should not automatically be deleted because they may represent legitimate one-to-many relationships such as multiple placement opportunities. The final dataset should only be exported after these validation checks confirm that the integration is consistent with the analytical objective.